REPORT

COMP3006: Full-Stack Development Report 2026/27

Bobby Mannino

University of Plymouth, Student Number 10878582

Updated October 29, 2026

Contents

1 Links **1**

2 Introduction **1**

3 Functional Requirements **1**

4 Target Audience **2**

5 Existing Products **2**
5.1 Blogger . . . 3
5.2 WordPress . . . 4
5.3 Existing Products Comparison . . . 5

6 AI Declaration **5**

7 Architecture **6**
7.1 Architecture Topology . . . 6
7.2 Nginx . . . 7
7.3 Database . . . 8
7.4 Web UI . . . 8
7.5 API . . . 8

8 Design Principles **8**

9 Structures **9**
9.1 Database Structures . . . 9
9.2 API Endpoints . . . 11

10 Testing **12**
10.1 Isolated Testing . . . 12
10.2 Holistic Testing . . . 12

11 Continuous Integration/Deployment **12**

12 Evaluation **13**

References **14**

List of Figures

1 Blogger 3
2 WordPress 4
3 Architecture Topology 7
4 Entity Relationship Diagram 10

List of Tables

1 Functional Requirements 1
2 Existing Products Comparison 5
3 API Endpoints 11

1 Links

GitHub Repository: github.com/bobbymannino/comp3006

YouTube Video: *TODO*

2 Introduction

The project I have decided to pursue is making a blog. The product shall be called **Bloggy**. The key priorities for this product are for it to work, have a simple UX, and to be performant.

3 Functional Requirements

Table 1 Functional Requirements

ID	Requirement
FR01	Users can create an account
FR02	Users can edit or delete their own accounts
FR03	Users can create/update/delete their own posts
FR04	Users can like their own and other peoples posts
FR05	Users can publish/unpublish their posts
FR06	The feed page will be live meaning new posts will be shown as they are published
FR07	Any user can see all published posts

Because I have prior knowledge in this specific field (web/API/database), I do not have a priority list for the functional requirements. I'm confident I can deliver all the features to a high standard on time.

4 Target Audience

There is no specific demographic or user base in mind for this project, it is designed for any user. A high priority for this project is **usability** and **efficiency**. I would like the people who use this to know instantly without any training or documentation how to use the product, as well as the product feeling fast to use.

5 Existing Products

There are several existing products that provide similar functionality to this project. Some of the most popular ones include:

5.1 Blogger

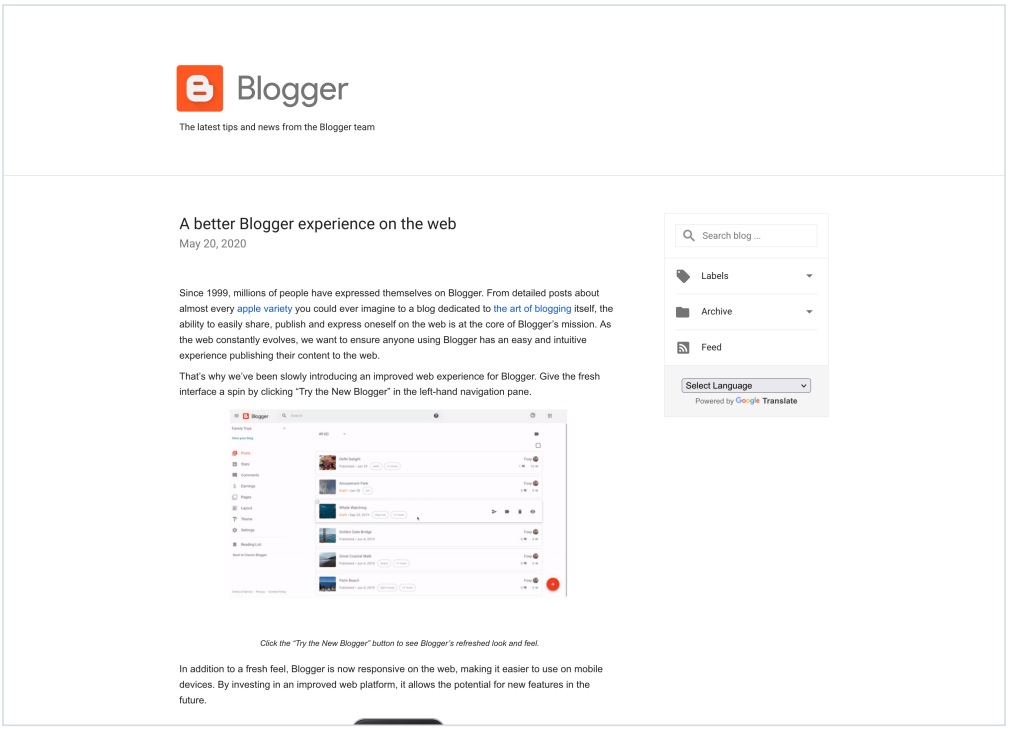


Figure 1 Blogger

I have not used Blogger in over 5 years, but when I was using it for school it got the job done. I will take inspiration from Blogger from a UX perspective, keeping things simple and easy to understand was and is Bloggers best feature.

5.2 WordPress

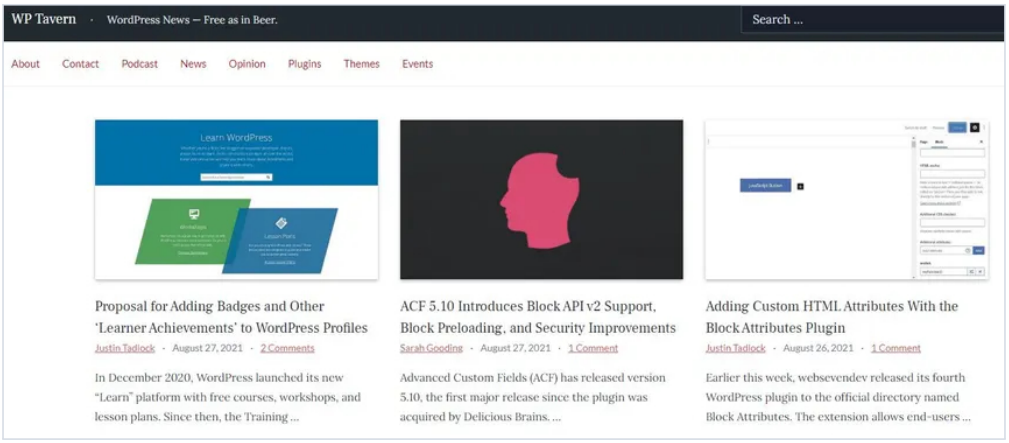


Figure 2 WordPress

I've never used WordPress before, but I know it has a massive market share online. I also know it's more than a blog; it can do nearly anything, including customizing the look. That customization is beyond this project's scope. I'll use WordPress's complexity as a warning: there are many plugins, and while they help, they can be overwhelming for someone wanting a straightforward public blog.

5.3 Existing Products Comparison

Table 2 Existing Products Comparison

Product	Advantages	Disadvantages
Blogger	• Simple, easy to understand UI • Free • No setup or hosting needed	• Limited customisation • Dated look • Few updates or new features
WordPress	• Huge market share and community • Highly customisable • Many plugins	• Overwhelming for simple blogs • Plugins add complexity • Self-hosting needed
Bloggy	• Simplest UI/UX • Performant	• No extensions • Less features • Tiny userbase

6 AI Declaration

In this project I have used AI. It has been used to create a huge amount of seeding data for the database, this makes the app feel more realistic and less like a generic blog platform. It has also been used to speed up the creation of tests, I want to have as many unit and integration tests as possible and so I have used AI to add as many as it can think on top of mine.

7 Architecture

7.1 Architecture Topology

The product will follow a microservices architecture. There will be 4 Docker images in this project:

- Nginx
- Database
- API
- Web UI

Each one runs independently and communicate with each other, for instance to access the database you must use the API, to use the web UI you must use the Nginx server. Putting the API and Web UI into their own containers allows you to scale them independently and update them separately. If traffic becomes API heavy you can scale those up, or if the user wants to host the API without having any UI they can do that without any changes to the API container.

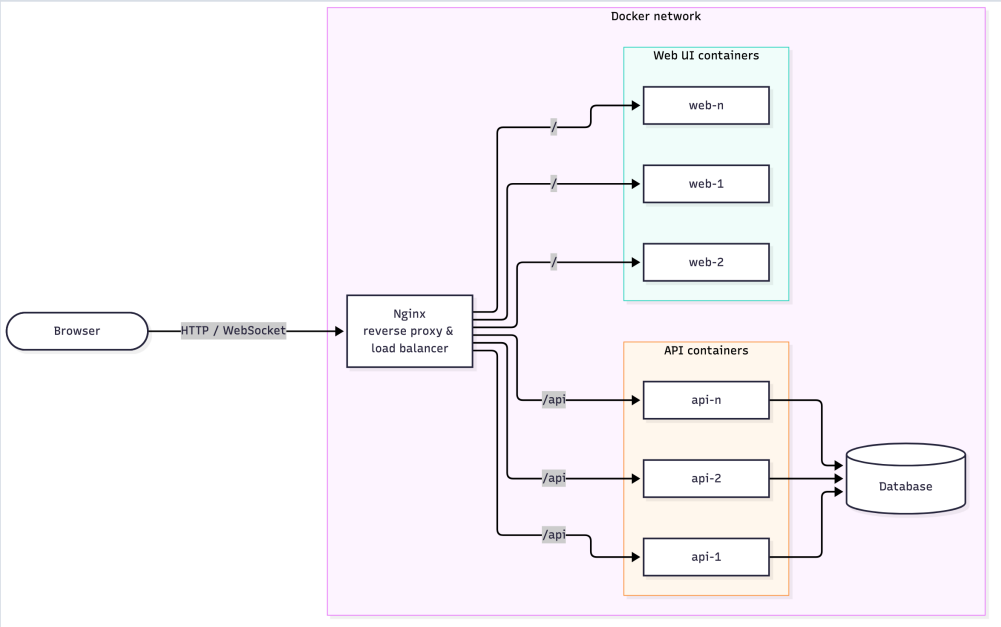


Figure 3 Architecture Topology

> **NOTE**
>
> Whilst it is possible to have multiple databases (read replicas) that is out of scope for this project so it will remain a single container.

7.2 Nginx

This image will not contain much custom code. There will be a configuration file to point specific paths to containers, a load balancer setup and that will be it. Nginx comes with enough boilerplate to handle most of what I want to achieve.

7.3 Database

The database will be a MongoDB container. It will be used to store all the persistent data from the blog, there will be no Redis or other cache layer as that is out of scope for this project. The database container will only be accessible to other containers on the same host machine, for this project it will only be accessed by the API image.

7.4 Web UI

The web UI image will be responsible for any interactions the user has. It will provide the user with a simple, intuitive interface to interact with the blog.

7.5 API

The API image is where the business logic of the blog is implemented. It will handle all the requests from the web UI and the database. It will be able to work without the web UI for people who want to build their own interfaces.

The latest version of the API image is available on Docker Hub: https://hub.docker.com/r/bobbymannino/comp3006-api.

8 Design Principles

My favorite principles are DRY and KISS, if you stick to these I believe your product will always be better than if you do not. KISS (keep it simple, stupid) can apply to both

UI and the source code. Keeping your code simplified, structured, and maintainable is key.

DRY (don't repeat yourself) is the act of not repeating any code. If the code is used 2 or more times then you should be separating that part of the code. This helps maintainability and can reduce bugs.

9 Structures

9.1 Database Structures

The database will hold four collections: `users`, `posts`, `likes` and `sessions`. A user can write many posts, and a like links one user to one post, which gives the many-to-many "users like posts" relationship needed for *FR04*. A user can also have many sessions, one for each time they log in.

Each collection's fields are shown below. MongoDB does not enforce foreign keys, so the `FK` fields are references the API is responsible for keeping valid.

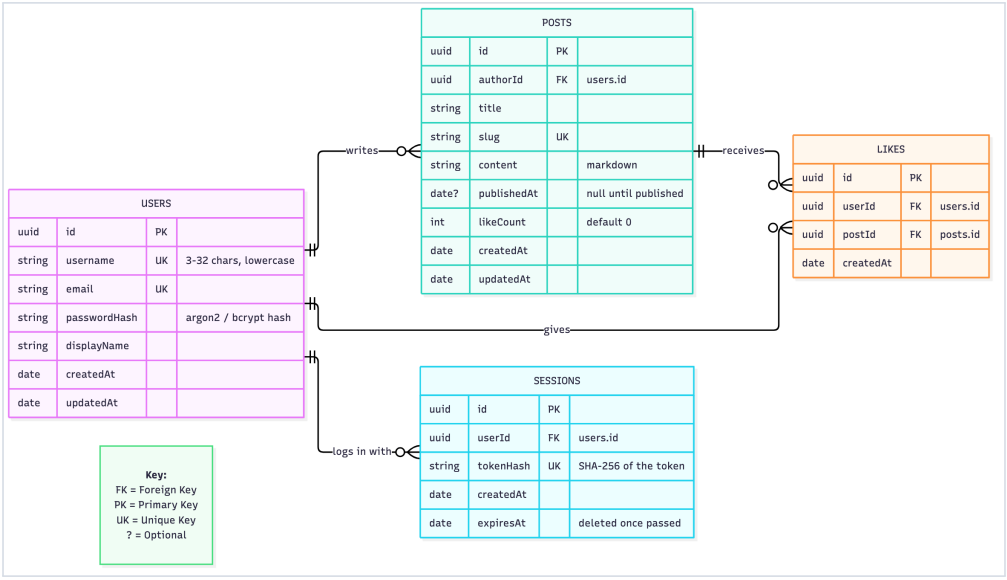


Figure 4 Entity Relationship Diagram

- **Users** — `username` and `email` have unique indexes so two accounts cannot share either. Only a hash of the password is stored, never the password itself.
- **Posts** — `published` and `publishedAt` cover FR05; unpublished posts are only visible to their author. `likeCount` is a copy of the number of likes so the feed does not need to count the `likes` collection for every post.
- **Likes** — a compound unique index on `userId` and `postId` stops a user liking the same post twice. Unliking a post deletes the document.
- **Sessions** — logging in creates a session and gives the user a random token to send as a bearer token. Only a SHA-256 hash of the token is stored, so a leaked database cannot be used to log in as anyone. Each session expires after 3 days and a TTL index on `expiresAt` has MongoDB delete it once it has.

> **NOTE**
>
> When a user deletes their account (FR02) the API will also delete their sessions, their posts, the likes on those posts and the likes they have given, so no documents are left pointing at a user that no longer exists.

When using MongoDB there is a feature to have an *init* scripts[1]. This allows for 2 things that I have used: firstly you can have a schema creation file to make sure that the database schema is created when the container starts, and secondly you can have a seed file to populate the database with some initial data.

9.2 API Endpoints

> **NOTE**
>
> All paths will be prepended with `/api`.

Table 3 API Endpoints

Method	Path	Action	Requires Bearer
PUT	`/users/:id`	Update a user by id	Y
DELETE	`/users/:id`	Delete a user	Y
GET	`/posts`	Get all posts	N
WS	`/posts`	Get all posts live	N
POST	`/posts`	Create a new post	Y
GET	`/posts/:id`	Get a post by id	N
PUT	`/posts/:id`	Update a post by id	Y
DELETE	`/posts/:id`	Delete a post by id	Y
POST	`/posts/:id/likes`	Create a new like	Y
DELETE	`/posts/:id/likes`	Delete a like by user and post	Y
POST	`/auth/login`	Sign in with an existing account	N
POST	`/auth/signup`	Create a new user account	N

10 Testing

10.1 Isolated Testing

This testing will be isolated to each image, for instance API testing will be API specific and not interact with any other component (such as Nginx). This will be useful for rapid testing and isolated test suites for scoped changes.

TODO

10.2 Holistic Testing

Comprehensive testing will be carried out on the whole system together, including every part, and will make sure they all function as intended. The point of this is to ensure that all the components work together as expected.

TODO

11 Continuous Integration/Deployment

Using GitHub's automation platform **Actions**[2], I will add in continuous testing along with automated Docker image creation. The image creation will only apply to the **web UI and API** images.

12 Evaluation

TODO

References

1. https://www.mongodb.com/docs/atlas/cli/current/atlas-cli-docker-compose#initialization
2. https://github.com/features/actions